Image Steganography System

High Level Design Document

Prepared by

Mohamed Aziz Amri
Mohamed Dhia Ben Kilani
Elaa Chaaleb

April 2026

Contents

1. Introduction

This document presents the High Level Design (HLD) for the Image Steganography System. It describes the overall architecture, the core components, the data and message flows between them, the development toolchain, and the planned implementation phases.

The system enables users to embed secret messages inside ordinary images (a process called steganography) and to extract those messages again, all without making the image look any different to the human eye. The primary technique used is Least Significant Bit (LSB) substitution, with optional encryption for added security.

2. System Objectives

- Allow a sender to hide a text message (or small file) inside a cover image and produce a stego-image that is visually identical to the original.
- Allow a receiver to extract the hidden message from the stego-image using a matching key or password.
- Maintain imperceptibility: changes to pixel values must not be visible to the naked eye.
- Provide optional encryption of the message before embedding to improve security.
- Expose a clean desktop/web interface that requires no background knowledge in security from the end user.
- Deliver a modular, testable codebase that can be extended with stronger algorithms in the future.

3. High Level Architecture

The system is structured into three layers: a Presentation Layer (what the user sees), a Business Logic Layer (the algorithms and processing), and a Data / Storage Layer (files in and out). The table below summarises how these layers relate.

Architecture Overview

Layer	Responsibility	Key Components
Presentation Layer	User interface — forms, file upload, result display	Frontend (HTML/JS or Python GUI)
Business Logic Layer	Encoding & decoding pipeline, encryption, validation	Encoder Module, Decoder Module, Crypto Module, Validator
Data / Storage Layer	Image I/O, temporary file handling, output generation	Image I/O Handler, File Manager, Output Generator

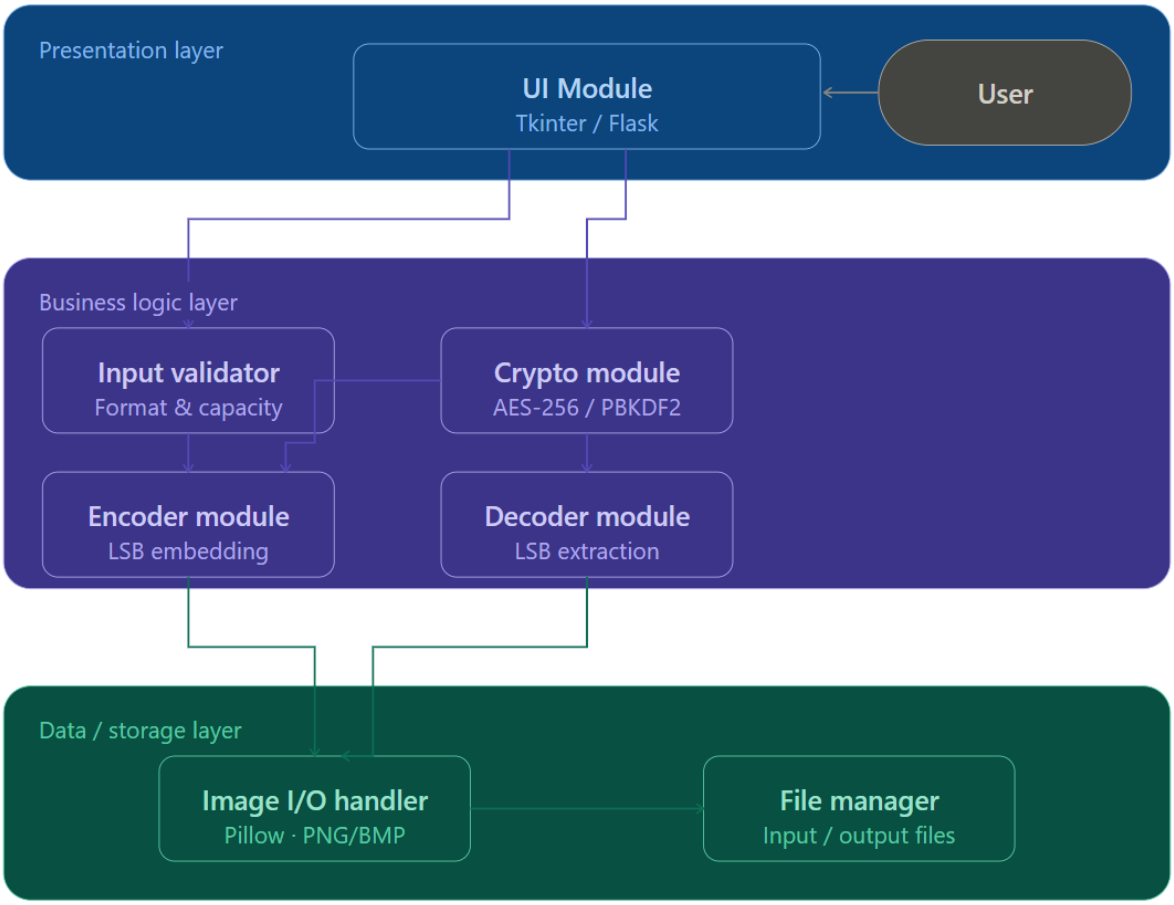

Figure 1: System Architecture Diagram — three-layer component overview

4. Component Descriptions

The system is composed of six primary components. Each is described below.

4.1 User Interface (UI) Module

The front-facing layer that the user interacts with. Responsibilities:

- Accept a cover image (PNG or BMP preferred) from the user.
- Accept a secret message as plain text input or as a small file.
- Accept an optional encryption password.
- Display the resulting stego-image for download (encoding mode).
- Accept a stego-image and password; display the extracted message (decoding mode).
- Show error messages for unsupported formats, oversized messages, or wrong passwords.

4.2 Input Validator

A defensive layer that sits between the UI and the processing modules. Responsibilities:

- Verify that the uploaded image is a supported format (PNG, BMP).
- Calculate the maximum message capacity for the given image (pixels $\times$ 3 channels $\times$ 1 LSB bit / 8 = bytes).
- Reject messages that exceed the available capacity.
- Sanitise text input to prevent injection or encoding issues.

4.3 Crypto Module

Handles optional symmetric encryption of the message before embedding and decryption after extraction. Responsibilities:

- Encrypt plaintext message using AES-256 (CBC mode) with a key derived from the user password via PBKDF2.
- Prepend an initialisation vector (IV) and a magic header to the ciphertext so the decoder can recognise and decrypt it.
- Decrypt the payload extracted by the Decoder and return the original plaintext.
- If no password is provided, pass the message through unmodified.

4.4 Encoder Module

The core encoding engine. Responsibilities:

- Accept the (optionally encrypted) binary payload.

- Convert the payload to a bit stream and prepend a length header (32 bits) so the decoder knows when to stop.
- Iterate over the cover image pixels in raster order and replace the least significant bit of each colour channel (R, G, B) with one payload bit.
- Produce the stego-image as a lossless PNG file.

4.5 Decoder Module

The core decoding engine. Responsibilities:

- Accept a stego-image.
- Read the LSB of each colour channel in raster order to reconstruct the bit stream.
- Parse the length header from the first 32 bits to know how many bytes to read.
- Pass the extracted byte array to the Crypto Module for decryption (if applicable).
- Return the recovered plaintext message to the UI.

4.6 Image I/O Handler

A thin utility layer that wraps the image library. Responsibilities:

- Open and decode image files into pixel arrays (using Pillow).
- Write modified pixel arrays back to lossless PNG files.
- Reject JPEG inputs automatically (lossy compression destroys LSB data).

5. Data and Message Flows

5.1 Encoding Flow

The sequence of steps and data passed between components when hiding a message:

Step	From	To	Data Exchanged
1	User	UI Module	Cover image file + secret message + password (optional)
2	UI Module	Input Validator	Image file path, message string, password flag
3	Input Validator	UI Module	Validation result (OK / error + reason)
4	UI Module	Crypto Module	Plaintext message + password
5	Crypto Module	Encoder Module	Encrypted binary payload (bytes)
6	Encoder Module	Image I/O Handler	Request to open cover image

continued on next page. . .

Step	From	To	Data Exchanged
7	Image I/O Handler	Encoder Module	Pixel array (NumPy array, H × W × 3)
8	Encoder Module	Image I/O Handler	Modified pixel array
9	Image I/O Handler	UI Module	Stego-image file (PNG)
10	UI Module	User	Download link for stego-image

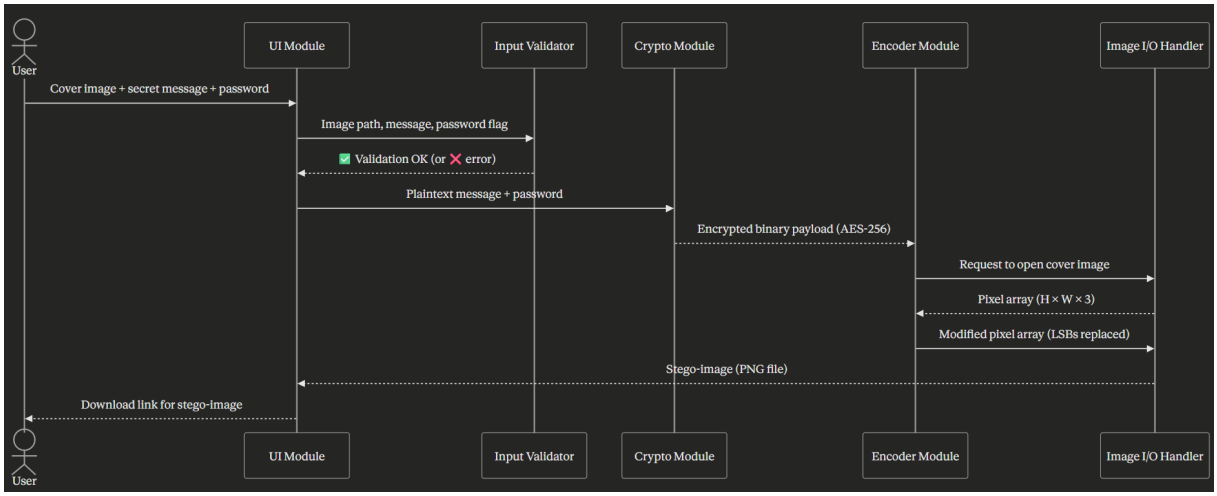

Figure 2: Encoding Flow — sequence diagram

5.2 Decoding Flow

The sequence of steps and data passed between components when recovering a hidden message:

Step	From	To	Data Exchanged
1	User	UI Module	Stego-image file + password (optional)
2	UI Module	Input Validator	Stego-image file path
3	Input Validator	UI Module	Validation result (OK / error)
4	UI Module	Decoder Module	Stego-image file path
5	Decoder Module	Image I/O Handler	Request to open stego-image
6	Image I/O Handler	Decoder Module	Pixel array (H × W × 3)

continued on next page. . .

Step	From	To	Data Exchanged
7	Decoder Module	Crypto Module	Raw extracted byte payload + password
8	Crypto Module	UI Module	Decrypted plaintext message
9	UI Module	User	Displayed secret message

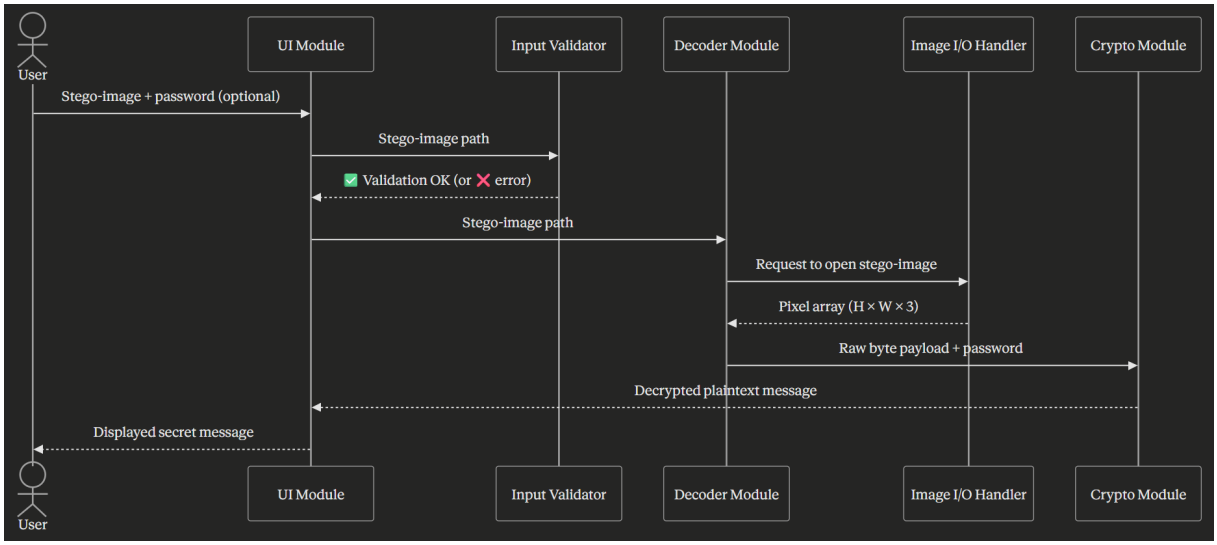

Figure 3: Decoding Flow — sequence diagram

5.3 Data Format — Embedded Payload Structure

The binary payload embedded inside the image follows a fixed structure to support reliable extraction:

Field	Size	Description
Magic Header	4 bytes	Fixed value `0x53544547` (`'STEG'`) — confirms a payload is present
Encryption Flag	1 byte	`0x01` if AES-encrypted, `0x00` if plaintext
Payload Length	4 bytes	Unsigned 32-bit integer: number of data bytes that follow
IV (if encrypted)	16 bytes	AES initialisation vector; omitted when flag is `0x00`
Data	N bytes	The encrypted ciphertext or raw UTF-8 message

6. Tools and Technologies

Category	Tool / Library	Version	Purpose
Language	Python	3.11+	Primary implementation language
Image Processing	Pillow (PIL)	10.x	Open, read, write PNG/BMP pixel data
Numerical Ops	NumPy	1.26+	Fast pixel array manipulation for LSB ops
Encryption	PyCryptodome	3.20+	AES-256-CBC encryption and PBKDF2 key derivation
UI (Desktop)	Tkinter	Built-in	Cross-platform desktop GUI for encode/decode forms
UI (Web alt.)	Flask	3.x	Lightweight web interface (optional alternative to Tkinter)
Testing	pytest	8.x	Unit and integration testing
Code Quality	flake8 / black	Latest	Linting and auto-formatting
Version Control	Git + GitHub	Latest	Source control and team collaboration
Documentation	Sphinx / Markdown	Latest	API docs and project readme

7. Development Phases

The project is divided into four phases. Each phase has a clear deliverable that can be demonstrated and tested independently.

Phase 1 — Core Engine (Foundation)

Deliverable: Working encode/decode pipeline for plain text, no encryption, no UI.

1. Set up the Git repository and project folder structure.
2. Implement Image I/O Handler: open PNG/BMP files, expose pixel arrays, save modified arrays.
3. Implement Encoder Module: convert string to bits, write LSBs, save stego-image.
4. Implement Decoder Module: read LSBs, reconstruct bits, recover string.
5. Write unit tests for encoder and decoder (round-trip test with known strings).
6. Validate that the stego-image is visually indistinguishable from the cover image (PSNR $>$ 40 dB).

Phase 2 — Security Layer

Deliverable: Optional AES-256 encryption of the message before embedding.

1. Implement Crypto Module: AES-256-CBC encrypt with PBKDF2 key derivation.
2. Embed the magic header, encryption flag, and IV in the payload structure.
3. Update Decoder Module to detect the flag and invoke Crypto Module for decryption.
4. Implement Input Validator: format check, capacity check, password strength hint.
5. Write security tests: wrong password should return an error, not garbled text.

Phase 3 — User Interface

Deliverable: Desktop GUI (Tkinter) with full encode and decode workflows.

1. Design and build the Tkinter UI: two-tab layout (Encode / Decode).
2. Wire UI events to the Validator, Crypto, Encoder, and Decoder modules.
3. Add progress indicators and user-friendly error dialogs.
4. Conduct usability testing with a team member who has no technical background.
5. (Optional) Build a Flask web UI as an alternative interface.

Phase 4 — Testing, Hardening & Documentation

Deliverable: Tested, documented, packaged release.

1. Run full integration test suite (cover images of different sizes, messages of different lengths, with and without password).
2. Perform basic steganalysis using zsteg and StegExpose to measure detectability.
3. Profile performance: measure encode/decode time for a 1 MP image.
4. Fix any bugs discovered during testing.
5. Write the final README, user guide, and API documentation (Sphinx).
6. Package the project as a zip / installer for submission.

8. Development Timeline Summary

Phase	Activities
Phase 1 — Core Engine	Image I/O, Encoder, Decoder, unit tests
Phase 2 — Security Layer	Crypto module, payload structure, validation
Phase 3 — User Interface	Tkinter GUI, event wiring, UX testing

continued on next page. . .

Phase	Activities
Phase 4 — Testing & Docs	Integration tests, steganalysis, documentation

9. Key Risks and Mitigations

Risk	Impact	Mitigation
JPEG input by user	Lossy compression destroys hidden bits	Reject JPEG at validation step; accept PNG/BMP only
Message too large for image	Overflow / corrupted output	Capacity check in Validator before embedding
Wrong password at decode	Garbled output or crash	Magic header check + exception handling in Crypto Module
LSB detectable by steganalysis tools	Privacy risk in high-stakes use	Document limitation; plan to add spread-spectrum LSB in future
Team unfamiliarity with crypto APIs	Bugs in encryption logic	Use PyCryptodome (well-documented); write thorough unit tests

10. Conclusion

This High Level Design document defines a clean, modular architecture for the Image Steganography System. The six components — UI Module, Input Validator, Crypto Module, Encoder Module, Decoder Module, and Image I/O Handler — interact through well-defined data flows that keep each concern separate and independently testable.

The four development phases provide a clear path from a working core engine to a polished, secure, and user-friendly application. All tools chosen (Python, Pillow, NumPy, PyCryptodome, Tkinter) are open-source, well-documented, and appropriate for a security-focused academic project.

The design deliberately keeps the interface simple so that users without a security background can use the system confidently, while the underlying encryption layer ensures that even if the stego-image is intercepted, the hidden message remains protected.